Image Processing

Lecture 3: Arithmetic and Logical Operations

By

Dr. Rasha Mahmoud Kamel

Operations Upon Pixels

- The most basic type of image processing operation is ***a point transform*** which maps the values at individual points (i.e. pixels) in the input image to corresponding points (pixels) in an output image.
- In the mathematical sense, this is a ***one-to-one functional mapping from input to output***.
- The simplest examples of such image transformations are ***arithmetic or logical operations on images.***
- Each is performed as an operation ***between two images or between an image and a constant value (scalar).***

Arithmetic Operations on Images

- **Arithmetic operations** are typically ***performed on a pixel-by-pixel basis.***
- That is, ***the operation is independently applied to each pixel in the image.***
- Given an an image I_A and another image I_B or a scalar C, the resulting image I_{Output} is obtained by calculating:

$$I_{\text{Output}} = I_A \ \ \mathbf{Opr} \ \ I_B$$

$$I_{\text{Output}} = I_A \ \ \mathbf{Opr} \ C$$

where $\mathbf{Opr}$ is an arithmetic operator (e.g. +, −, ×, /) .

- Note that the two images must be of the same size.

Arithmetic Operations on Images

- In both cases, ***the values at an individual pixel location*** (i, j) in the output image are mapped as follows:

$$I_{\text{Output}}(i,j) = I_A(i,j) \;\; \mathbf{Opr} \;\; I_B(i,j)$$

$$I_{\text{Output}}(i,j) = I_A(i,j) \;\; \mathbf{Opr} \;\; C$$

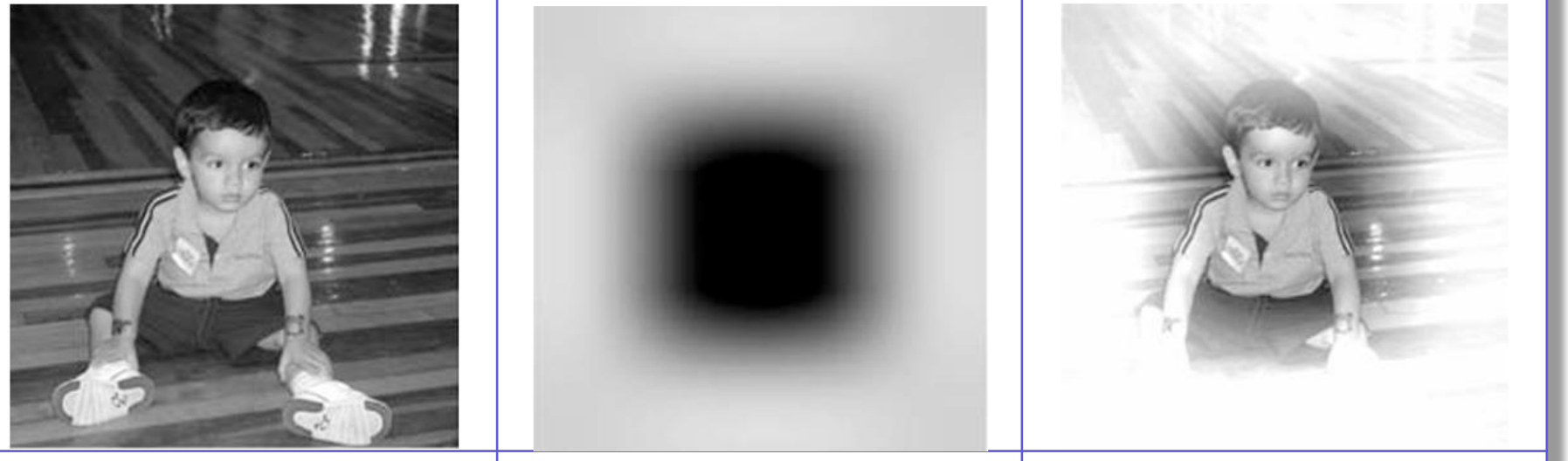
- To perform the operation over an entire image of dimension $M \times N$, ***we simply iterate over all image indices*** for $(i, j) = \{0 \cdots M-1, 0 \cdots N-1\}$ in the general case (in Matlab pixel indices are $\{1 \cdots M\,, 1 \cdots N\}$).

- Basic arithmetic operations can be performed quickly and easily on image pixels for a variety of effects and applications.

Image Addition

- ***Adding a value to each image pixel value*** can be used to achieve the following effects:
  - **Blending:** ***Adding images together produces a composite image of both input images.***

    This can be used to produce blending effects using weighted addition.



(a)	(b)	(c)

Adding two images: (a) first image (I_A); (b) second image (I_B); (c) result ($I_{\text{Output}} = I_A + I_B$)

Image Addition

- **Contrast adjustment:** ***Adding a positive constant value C (scalar) to each pixel location increases its value and, hence, its brightness.*** A process sometimes referred to as **additive image offset.**

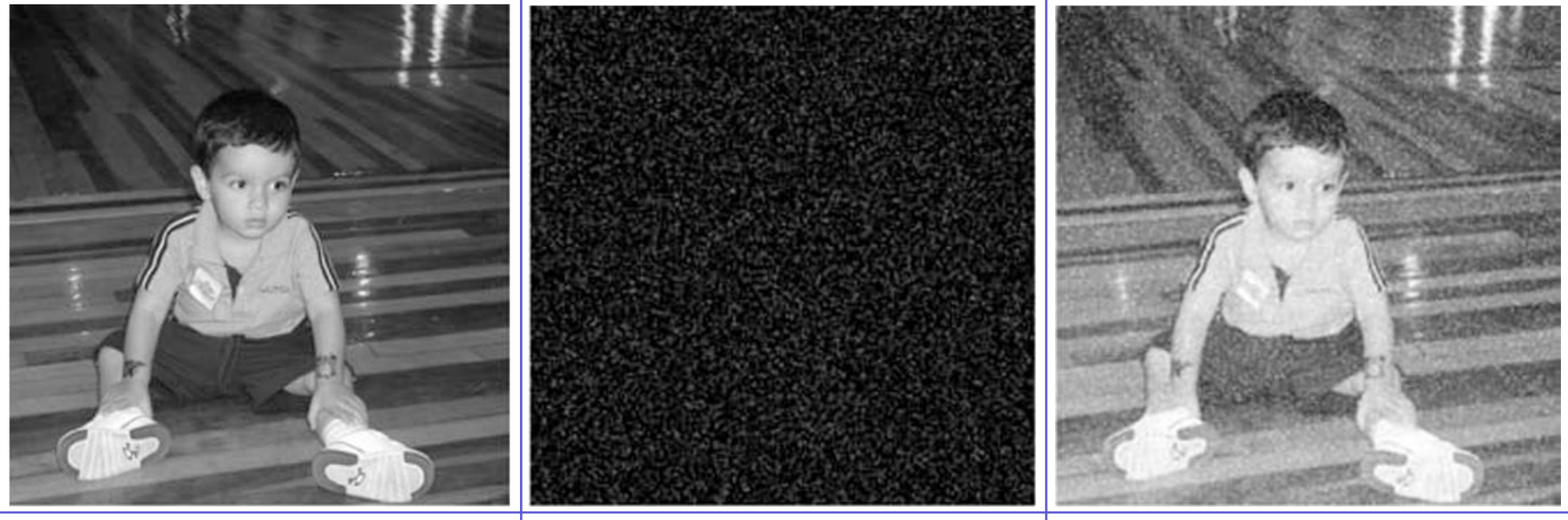
(a)

(b)

Additive image offset: (a) original image (I_A); (b) brighter version ($I_{\text{Output}} = I_A + 75$).

Image Addition

- **Noising:** ***Adding random amounts to each pixel value*** is a common way to simulate additive noise. The resulting (noisy) image is typically used as a test image for restoration algorithms.



(a)	(b)	(c)

Adding noise to an image: (a) original image (I_A); (b) zero-mean Gaussian white noise(variance=0.01) (N); (c) result ($I_{\text{Output}} = I_A + N$).

Image Addition – MATLAB

- **MATLAB's** Image Processing Toolbox (IPT) has a built-in function called **imadd** that can be used to add two images or add a constant (scalar) to an image.
- Generally, images must be of the ***same dimension and of the same data type*** (e.g. 8-bit integer) for addition to be possible between them.
- When performing addition operations, ***we must be aware of integer overflow*** (values that exceed the maximum pixel value for the datatype being used). An 8-bit integer image can only hold integer values from 0 to 255.
- There are ***two ways of dealing with this overflow issue:*** **truncation** and **normalization**.
- The Matlab **imadd** function ***avoids the overflow issue by truncating the values to the maximum value supported by the data type.***

Image Addition – MATLAB

- **Truncation** consists in simply ***limiting the results to the maximum positive number*** that can be represented with the adopted datatype.

- **Normalization** consists in ***storing the intermediate result in a temporary variable*** (I_{Int}) and calculating each resulting pixel value in I_{Output} using the following equation:

$$g = \frac{L_{max}}{f_{max} - f_{min}} (f - f_{min}) \quad *$$

where f is the current pixel in I_{Int}, L_{max} is the maximum possible intensity value (e.g., 255 for uint8 or 1.0 for double), g is the corresponding pixel in I_{Output}, f_{max} is the maximum pixel value in I_{Int}, and f_{min} is the minimum pixel value in I_{Int}.

Image Addition – MATLAB

- **Example:** For the two 3×3 monochrome images below (I_A and I_B), each of which represented as an array of unsigned integers, 8-bit (uint8), calculate $I_{\text{Output}} = I_A + I_B$, using **(a)** normalization and **(b)** truncation.

$$I_A = \begin{bmatrix} 200 & 100 & 100 \\ 0 & 10 & 50 \\ 50 & 250 & 120 \end{bmatrix}, I_B = \begin{bmatrix} 100 & 220 & 230 \\ 45 & 95 & 120 \\ 205 & 100 & 0 \end{bmatrix}$$

- **Solution:** The intermediate array I_{Int} (an array of unsigned integers, 16-bit, uint16) is obtained by simply adding the values of I_A and I_B on a pixel-by-pixel basis:

$$I_{Int} = \begin{bmatrix} 300 & 320 & 330 \\ 45 & 105 & 170 \\ 255 & 350 & 120 \end{bmatrix}$$

Image Addition – MATLAB

(a) Normalizing the [45,350] range to the [0,255] interval using equation (*), we obtain

$$I_{\text{Output}} = \begin{bmatrix} 213 & 230 & 238 \\ 0 & 50 & 105 \\ 176 & 255 & 63 \end{bmatrix}$$

(b) Truncating all values above 255 in I_{Int}, we obtain

$$I_{\text{Output}} = \begin{bmatrix} 255 & 255 & 255 \\ 45 & 105 & 170 \\ 255 & 255 & 120 \end{bmatrix}$$

Image Addition – MATLAB

- **MATLAB code:**

```
IA = uint8([200 100 100; 0 10 50;50 250 120]);
IB = uint8([100 220 230; 45 95 120;205 100 0]);
I_Int = uint16(IA) + uint16(IB);
F_max = max(I_Int(:));
F_min = min(I_Int(:));
I_Outa = uint8(255.0 * double(( I_Int - F_min )) / double(( F_max - F_min )));  %Normalization
I_Outb = imadd(IA,IB); %Truncation
```

Image Addition – MATLAB

- **Example:** Use the imadd function to brighten an image by adding a constant (scalar) value to all its pixel values.

```
Img = imread('tire.tif');
I_Out = imadd(Img,75);
figure
subplot(1,2,1), imshow(Img), title('Original Image');
subplot(1,2,2), imshow(I_Out), title('Brighter Image');
```

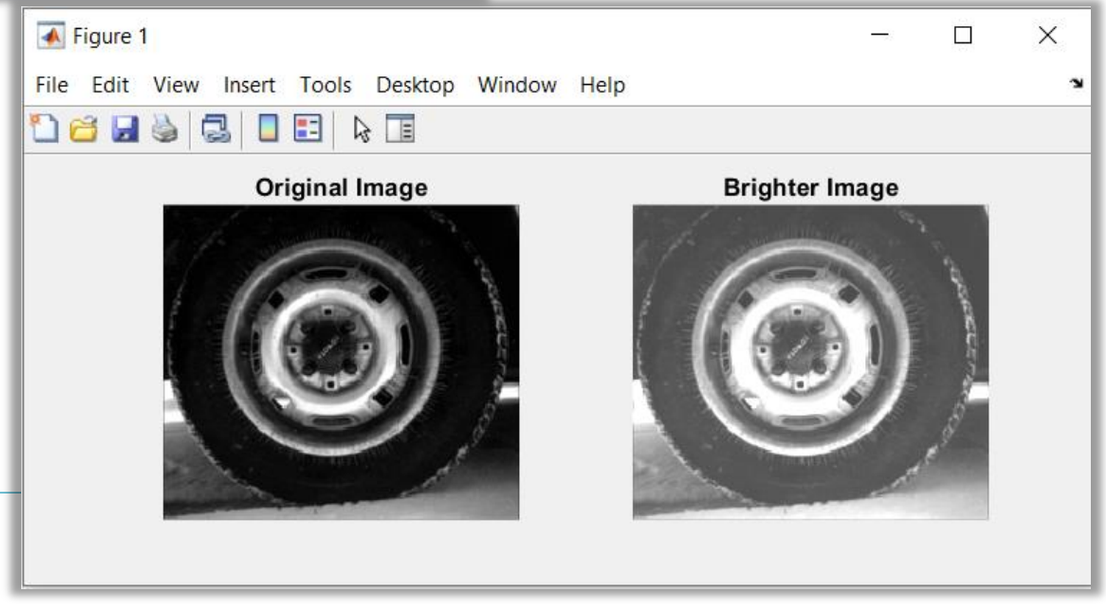


Image Addition – MATLAB

- **Example:** Use the imadd function to blend two images.

```
IA = imread('rice.png');
IB = imread('cameraman.tif');
I_Out = imadd(IA,IB);
figure
subplot(1,3,1), imshow(IA), title('First Image');
subplot(1,3,2), imshow(IB), title('Second Image');
subplot(1,3,3), imshow(I_Out), title('Output Image');
```

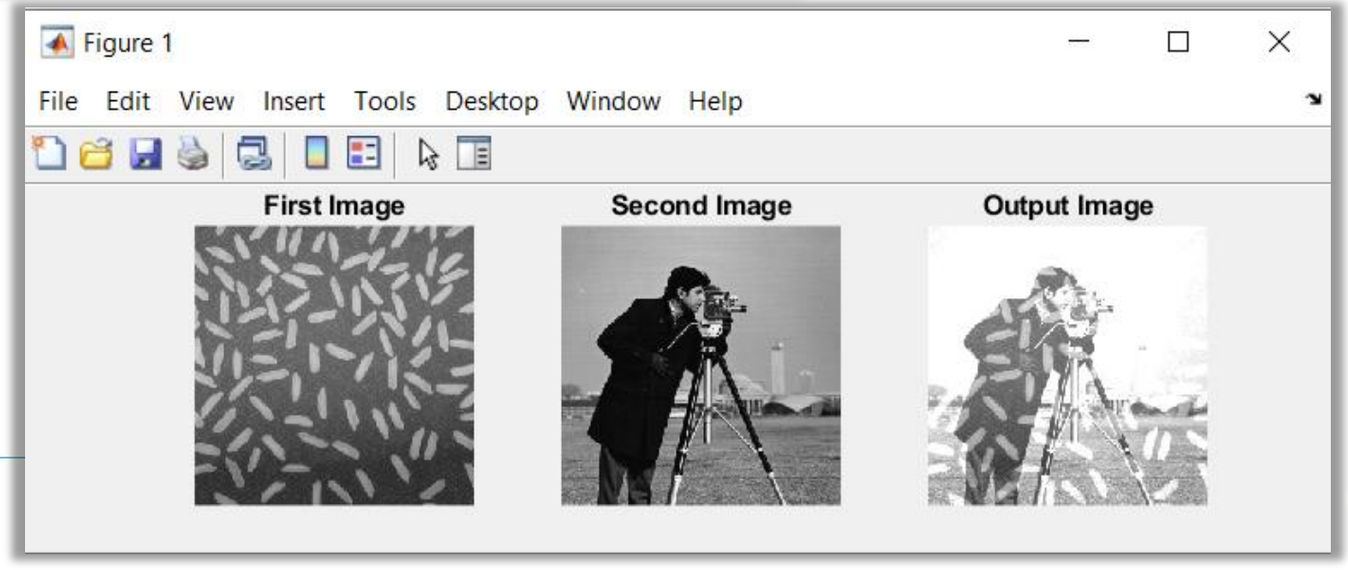


Image Subtraction

- **Subtraction** is often used ***to detect differences between two images.***
- Such differences may be due to several factors, such as ***artificial addition to or removal of relevant contents from the image*** (e.g., using an image manipulation program).
- ***If we subtract two images in a video sequence*** then we get a difference image (assuming a static camera) which shows the movement or changes that have occurred between the frames in the scene.
- ***Subtracting a constant value (scalar) from an image causes a decrease in its overall brightness***, a process sometimes referred to as **subtractive image offset.**

Image Subtraction

- When subtracting one image from another or a constant (scalar) from an image, ***you must be careful with the possibility of obtaining negative pixel values as a result.***
- There are two ways of dealing with this **underflow issue**: treating subtraction as ***absolute difference*** and ***truncating the result***.
- **Absolute difference** will always result in positive values proportional to ***the difference between the two original images without indicating which pixel was brighter or darker***.
- In **truncation**, ***negative values become zero***.

Image Subtraction – MATLAB

- **MATLAB's** IPT has a built-in function called **imsubtract** that can be used to subtract one image from another, or subtract a constant from an image.
- The Matlab function **imsubtract** ***avoid the underflow issue by truncating underflow values***. When we perform image subtraction, some of the pixels resulted in negative values, but the imsubtract function set these pixels to 0 (the function does this on purpose to keep the data within grayscale range).
- The IPT also has a built-in function called **imabsdiff** that can be used ***to calculate the absolute difference of two images.***

Image Subtraction – MATLAB

- **Example:** For the two 3×3 monochrome images below (I_A and I_B), each of which represented as an array of unsigned integers, 8-bit (uint8), calculate

  (a) $I_{\text{Output}} = I_A - I_B$ **(b)** $I_{\text{Output}} = I_B - I_A$ **(c)** $I_{\text{Output}} = |I_B - I_A|$

  For parts (a) and (b), use truncation to deal with possible negative values.

$$I_A = \begin{bmatrix} 200 & 100 & 100 \\ 0 & 10 & 50 \\ 50 & 250 & 120 \end{bmatrix}, I_B = \begin{bmatrix} 100 & 220 & 230 \\ 45 & 95 & 120 \\ 205 & 100 & 0 \end{bmatrix}$$

- **Solution:** MATLAB's imsubtract will be used for parts (a) and (b), while imabsdiff will be used for part (c).

Image Subtraction – MATLAB

(a)

$$I_{\text{Output}} = \begin{bmatrix} 100 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 150 & 120 \end{bmatrix}$$

(b)

$$I_{\text{Output}} = \begin{bmatrix} 0 & 120 & 130 \\ 45 & 85 & 70 \\ 155 & 0 & 0 \end{bmatrix}$$

(c)

$$I_{\text{Output}} = \begin{bmatrix} 100 & 120 & 130 \\ 45 & 85 & 70 \\ 155 & 150 & 120 \end{bmatrix}$$

- **MATLAB code:**

```
IA = uint8([200 100 100; 0 10 50;50 250 120]);
IB = uint8([100 220 230; 45 95 120;205 100 0]);
I_Outa = imsubtract(IA,IB);
I_Outb = imsubtract(IB,IA);
I_Outc =imabsdiff(IB,IA);
```

Image Subtraction – MATLAB

- **Example:** Use the imsubtract function to darken an image by subtracting a constant (scalar) value from all its pixel values.

```
IA = imread('cameraman.tif');
I_Out = imsubtract(IA,100);
figure
subplot(1,2,1), imshow(IA), title('Original Image');
subplot(1,2,2), imshow(I_Out), title('Darker Image');
```

Image Subtraction – MATLAB

- **Example:** Use the imsubtract function to highlight the differences between two images.

```
IA = imread('cameraman.tif');
IB = imread('cameraman2.tif');
I_Outa = imsubtract(IA,IB);
I_Outb = imabsdiff(IA,IB);
figure
subplot(2,2,1), imshow(IA), title('Original Image');
subplot(2,2,2), imshow(IB), title('Altered Image');
subplot(2,2,3), imshow(I_Outa,[]), title('Subtracted Image');
subplot(2,2,4), imshow(I_Outb,[]), title('Abs Diff Image');
```

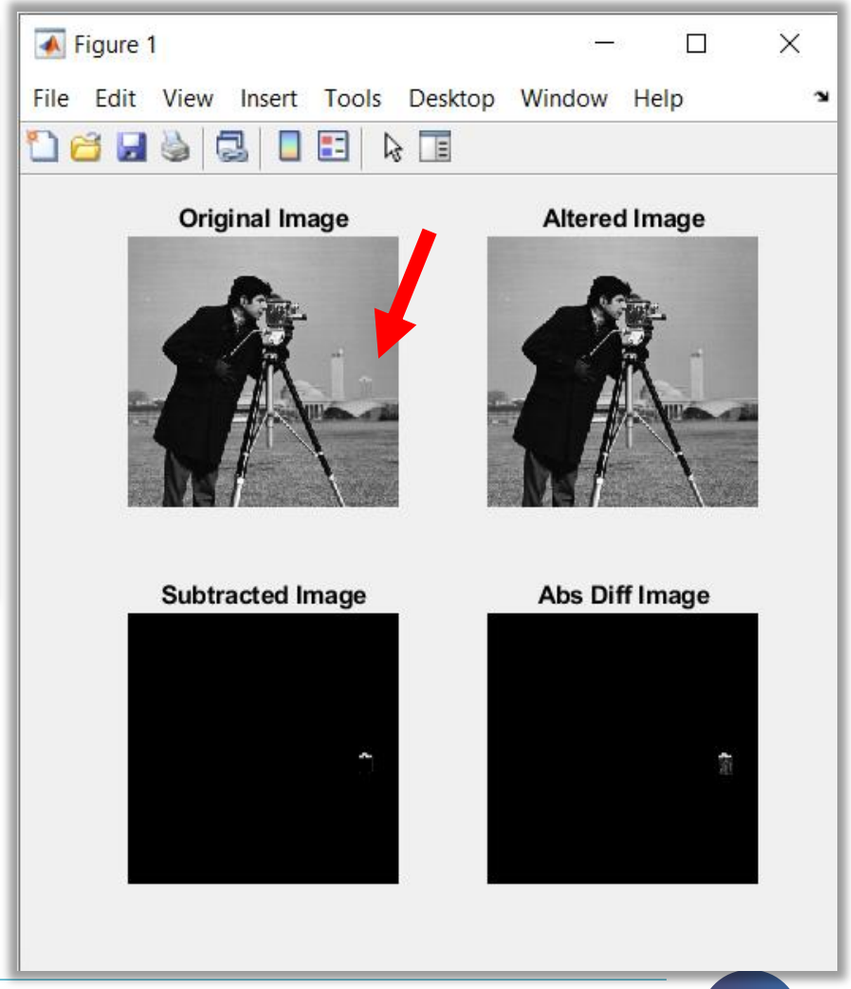


Image Multiplication

- **Multiplication** ***by a scalar can be used to perform contrast adjustment*** on an image (e.g., increasing contrast by 50% = multiplication by 1.5).
- This process—sometimes referred to as ***multiplicative image scaling***—makes each pixel value brighter (or darker) by multiplying its original value by a scalar factor:
  - If the value of the scalar factor is greater than one, the result is a brighter image.
  - If the value of the scalar multiplication factor is greater than zero and less than one, it results in a darker image.
- **Multiplicative image scaling** usually ***produces better subjective results than the*** **additive image offset process**.

Image Multiplication – MATLAB

- **MATLAB's IPT** has a built-in function called **immultiply** that can be used to multiply two images or multiply an image by a constant.
- **immultiply** function multiplies two images by multiplying the values of each pixel of same coordinates in both images.
- Multiplication can be used for a brightening process known as ***dynamic scaling***, which results in a more naturally brighter image compared to directly adding a constant to each pixel.
- Image multiplication can also be used for special effects such as an artificial 3D look. By ***multiplying a flat image with a gradient***, we create the illusion of a ***3D textured surface***.

Image Multiplication – MATLAB

- **Example:** Use immultiply to dynamically scale the moon image.

```
IA = imread('moon.tif');
I_Outa = imadd(IA,50);
I_Outb = immultiply(IA,1.2);
figure
subplot(1,3,1), imshow(IA), title('Original Image');
subplot(1,3,2), imshow(I_Outa), title('Normal Brightening');
subplot(1,3,3), imshow(I_Outb), title('Dynamic Scaling');
```

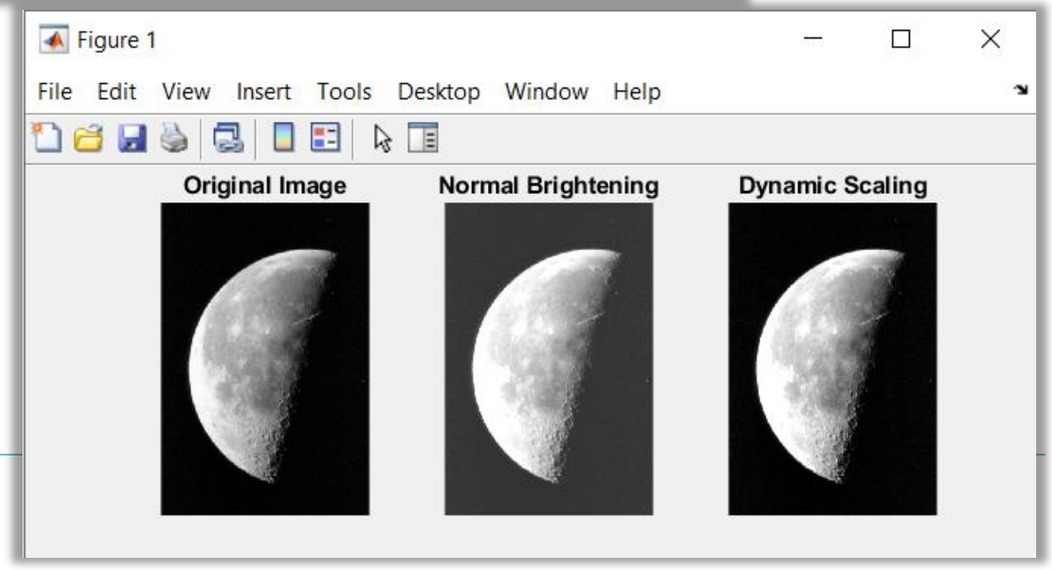


Image Multiplication – MATLAB

- **Example:** Create an artificial 3D planet using the earth1 and earth2 images.

```
IA = im2double(imread('earth1.tif'));
IB = im2double(imread('earth2.tif'));
I_Out = immultiply(IA,IB);
figure
subplot(1,3,1), imshow(IA), title('Planet Image');
subplot(1,3,2), imshow(IB), title('Gradient');
subplot(1,3,3), imshow(I_Out), title('3D Planet');
```

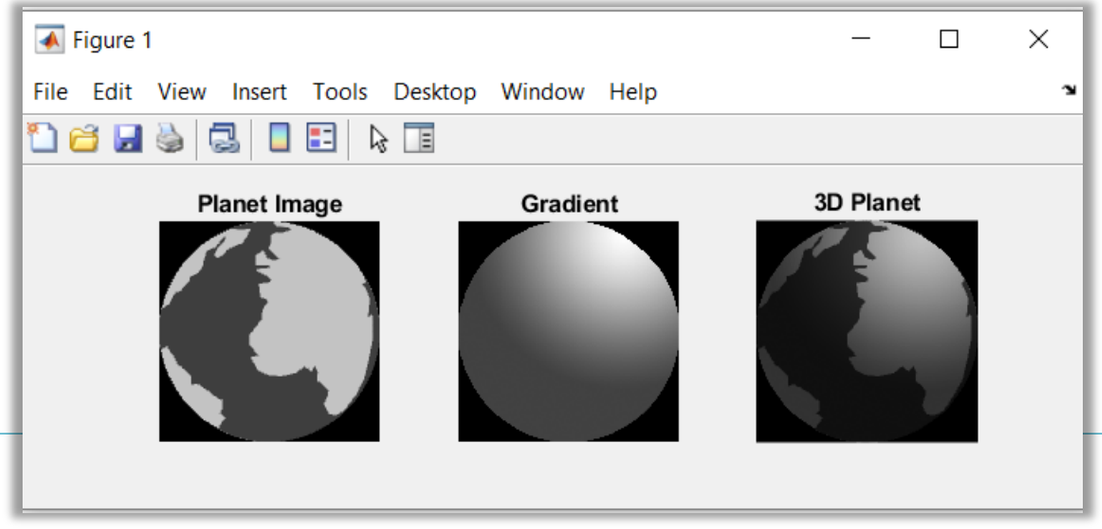


Image Division

- **Division** by a scalar can also be used to ***perform contrast adjustment on an image*** (e.g. Reducing contrast to 25% = division by 4).
- **Division** can be used for image differencing:
  - Dividing an image by another gives a result of 1.0 where the image pixel values are identical.
  - Dividing an image by another gives a value not equal to 1.0 where differences occur.
- However, ***image differencing using subtraction is computationally more efficient.***

Image Division – MATLAB

- **MATLAB's IPT** has a built-in function called **imdivide** that can be used to divide one image into another or divide an image by a constant.
- **Image division** can be used as ***the inverse operation to dynamic scaling***. That is, **imdivide** function is used to dynamically darken an image. However, we can achieve the same effect using the **immultiply** function.
- Another use of the image division process is to ***extract the background from an image***. This is usually done during a preprocessing stage of a larger, more complex operation.

Image Division – MATLAB

- **Example:** Use image division to dynamically darken the moon image.

```
IA = imread('moon.tif');
I_Outd = imdivide(IA,2);
I_Outm = immultiply(IA,0.5);
figure
subplot(1,3,1), imshow(IA), title('Original Image');
subplot(1,3,2), imshow(I_Outd), title('Darker Image Division')
subplot(1,3,3), imshow(I_Outm), title('Darker Image Multiplication');
% I_Outm is the equivalent darker image using image multiplication.
```

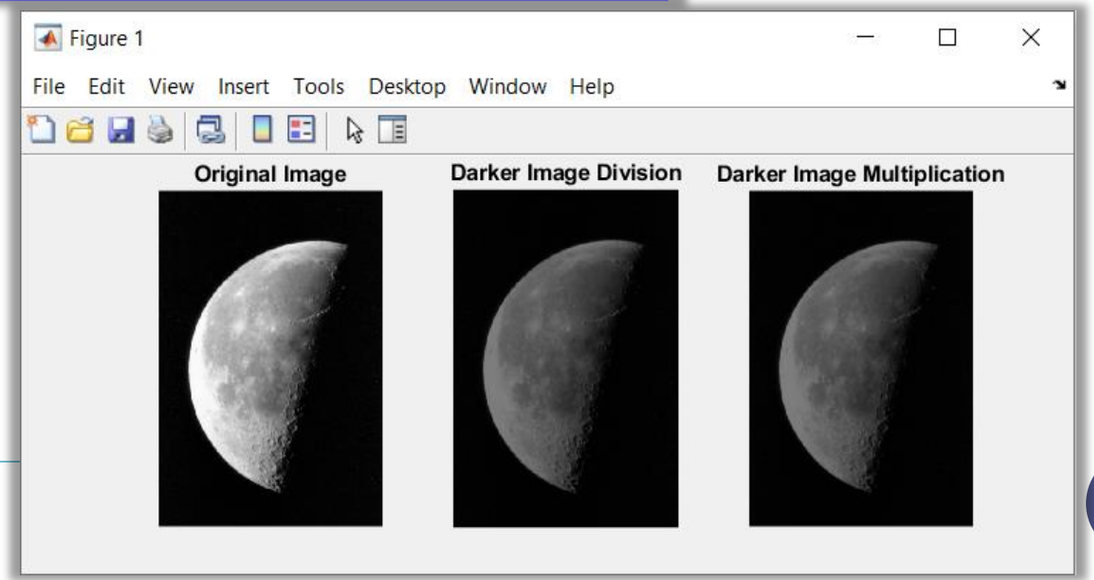
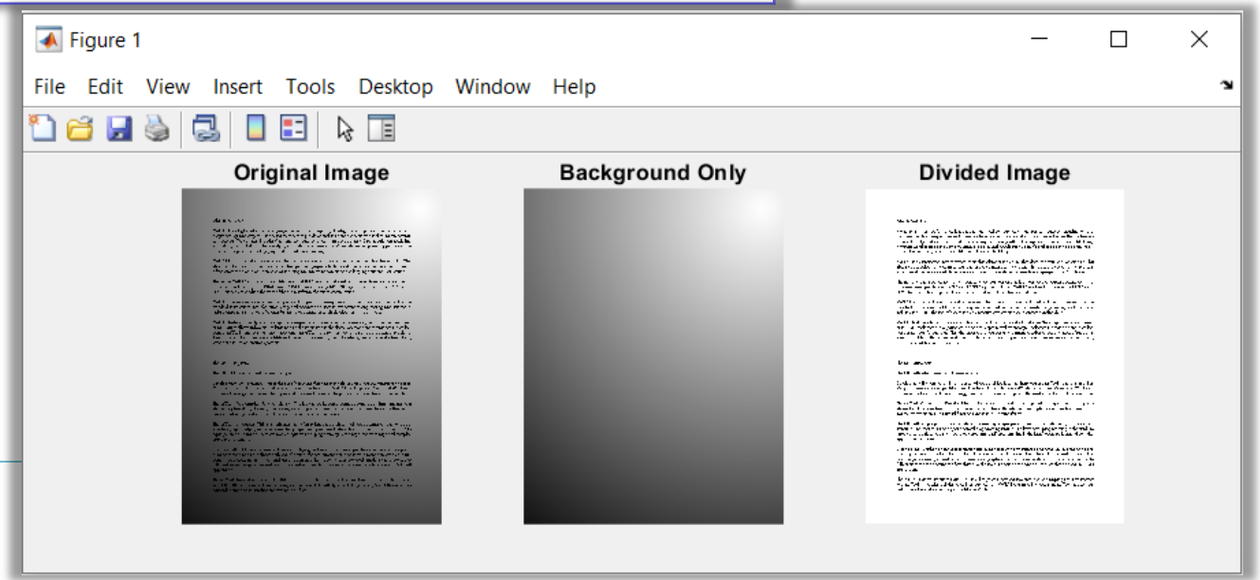


Image Division – MATLAB

- **Example:** Use division to divide the background from the image to get rid of the background.

```
notext = imread('gradient.tif');
text = imread('gradient_with_text.tif');
fixed = imdivide(text,notext);
figure
subplot(1,3,1), imshow(text), title('Original Image');
subplot(1,3,2), imshow(notext), title('Background Only');
subplot(1,3,3), imshow(fixed,[]), title('Divided Image')
```



Arithmetic Operations on Images

- For all arithmetic operations between images, we must ensure that the resulting pixel values remain within the available integer range of the datatype/size available.
- For instance, an 8-bit image (or three-channel 24-bit color image) can represent 256 values in each pixel location.
- An initial pixel value of 25 multiplied by a constant (or secondary image pixel value) of 12 will exceed the 0–255 value range. Integer overflow will occur and the value will ordinarily 'wrap around' to a low value.
- This is commonly known as **saturation in the image space**: the value exceeds the representational capacity of the image.

Arithmetic Operations on Images

- A solution is to detect this overflow and avoid it by setting all such values to the maximum value for the image representation (e.g. truncation to 255).
- This method of handling overflow is implemented in the imadd, imsubtract, immultiply and imdivide MATLAB functions.
- Similarly, we must also be aware of negative pixel values resulting from subtraction and deal with these accordingly; commonly they are set to zero.
- **For RGB images** (or other images with vectors as pixel elements) ***the arithmetic operation is generally performed separately for each color channel.***

Logical Operations on Images

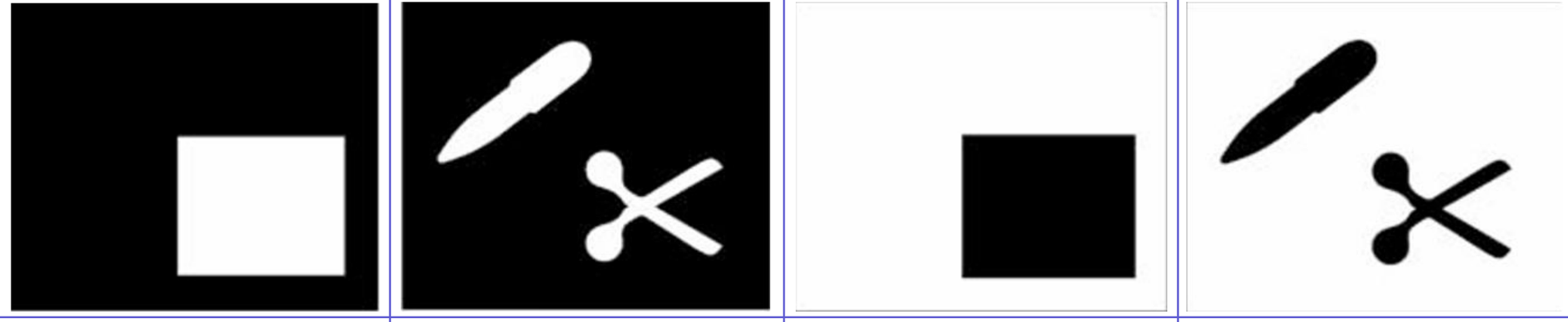
- We can perform standard logical operations between images such as NOT, OR, XOR and AND.
- In general, logical operation is performed between each corresponding bit of the image pixel representation (i.e. a bit-wise operator).
- The **AND, XOR**, and **OR** operators ***require two or more arguments***, whereas the **NOT** operator ***requires only one argument***.

Logical Operations on Images

- **NOT (inversion):** ***This inverts the image representation.*** It is equivalent to applying the "negative" effect on an image. In the simplest case of a binary image, the (black) background pixels become (white) foreground and viceversa. For greyscale and color images, the procedure is to replace each pixel value $I_{\text{Input}}(i,j)$ as follows:

$$I_{\text{Output}}(i,j) = MAX - I_{\text{Input}}(i,j)$$

where MAX is the maximum possible intensity value (e.g., 255 for uint8 or 1.0 for double).



I_A	I_B	**NOT** I_A	**NOT** I_B

Logical Operations on Images

- The **AND** and **OR operations** can be used to ***combine images*** for special effects purposes. They are also used in ***masking operations***, ***whose goal is to extract a region of interest (ROI) from an image.***

- The **XOR operation** is often used to ***highlight differences between two monochrome images***. It is, therefore,equivalent to calculating the **absolute difference** between two images.

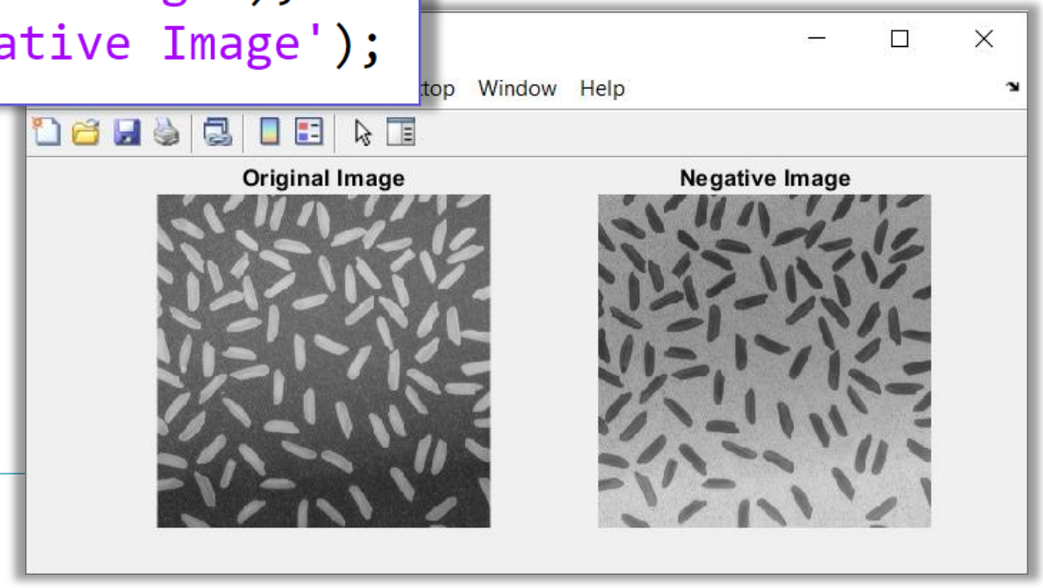
I_A	I_B	I_A **AND** I_B	I_A **OR** I_B	I_A **XOR** I_B

Logical Operations on Images – MATLAB

- The **MATLAB**'s IPT includes a function called **imcomplement** that can be used to calculate the negative of an image, complementing the image.

- **Example:**

```
IA = imread('rice.png');
I_Out = imcomplement(IA);
figure
subplot(1,2,1), imshow(IA), title('Original Image');
subplot(1,2,2), imshow(I_Out), title('Negative Image');
```

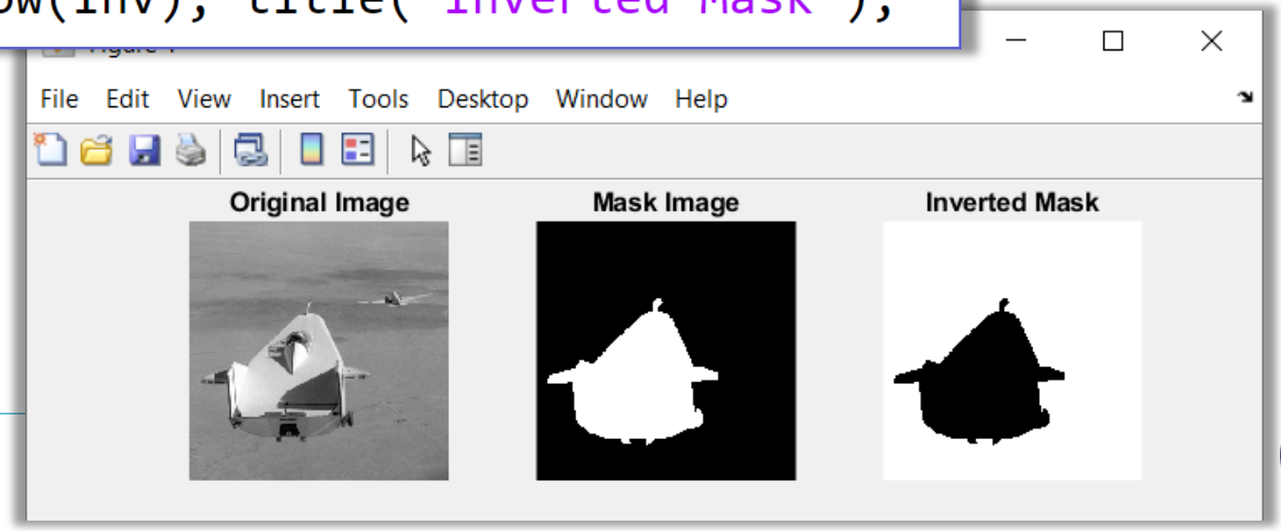


Logical Operations on Images – MATLAB

- The **MATLAB**'s IPT includes a function called **roipoly** that can be used to create the image mask. The generated mask will then be used to perform logic operations on the selected image.

- **Example:**

```
Img = imread('liftingbody.png');
Bin = roipoly(Img);
Inv = imcomplement(Bin);
figure
subplot(1,3,1), imshow(Img), title('Original Image');
subplot(1,3,2), imshow(Bin), title('Mask Image');
subplot(1,3,3), imshow(Inv), title('Inverted Mask');
```

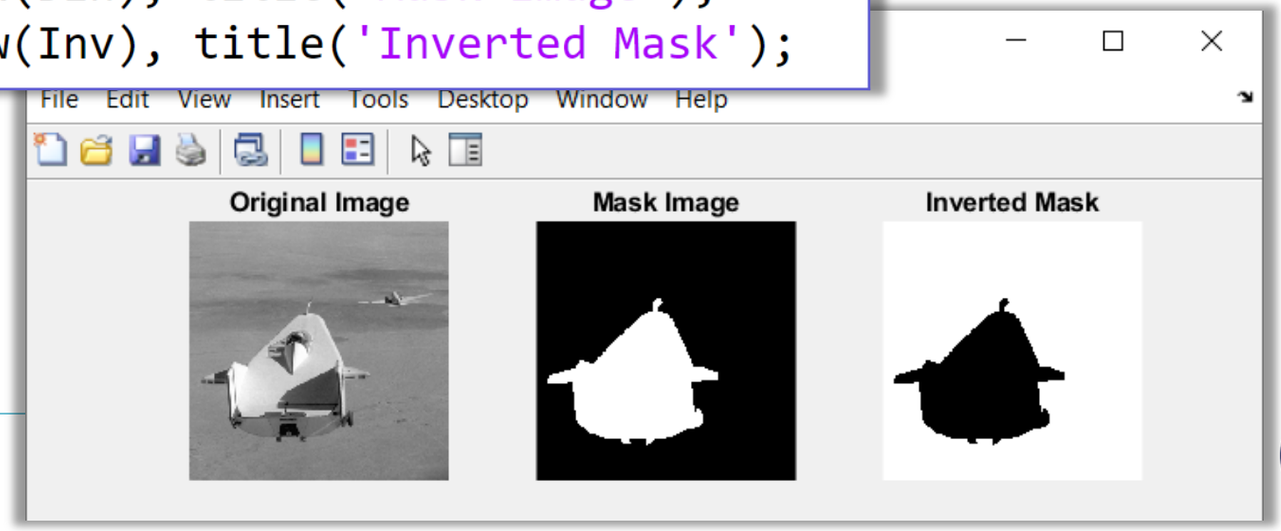


Logical Operations on Images – MATLAB

- The **MATLAB**'s IPT also includes function **bitcmp**, which performs the same operation as the **imcomplement** function. The imcomplement function allows input images to be binary, grayscale, or RGB, whereas bitcmp requires that the image be an array of unsigned integers.

- **Example:**

```
Img = imread('liftingbody.png');
Bin = roipoly(Img);
Inv = bitcmp(im2uint8(Bin));
figure
subplot(1,3,1), imshow(Img), title('Original Image');
subplot(1,3,2), imshow(Bin), title('Mask Image');
subplot(1,3,3), imshow(Inv), title('Inverted Mask');
```

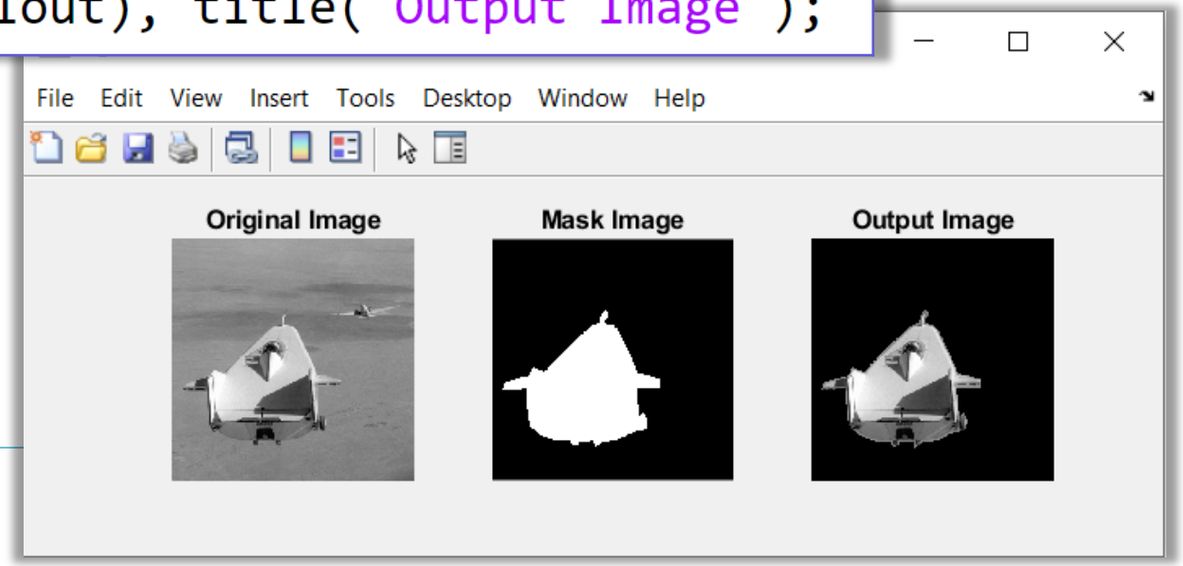


Logical Operations on Images – MATLAB

- The **MATLAB**'s IPT includes function called **bitand** that can be used to compute the logic AND between two images.

- **Example:**

```
Img = imread('liftingbody.png');
Mask = roipoly(Img);
Iout = bitand(Img, im2uint8(Mask));
figure
subplot(1,3,1), imshow(Img), title('Original Image');
subplot(1,3,2), imshow(Mask), title('Mask Image');
subplot(1,3,3), imshow(Iout), title('Output Image');
```

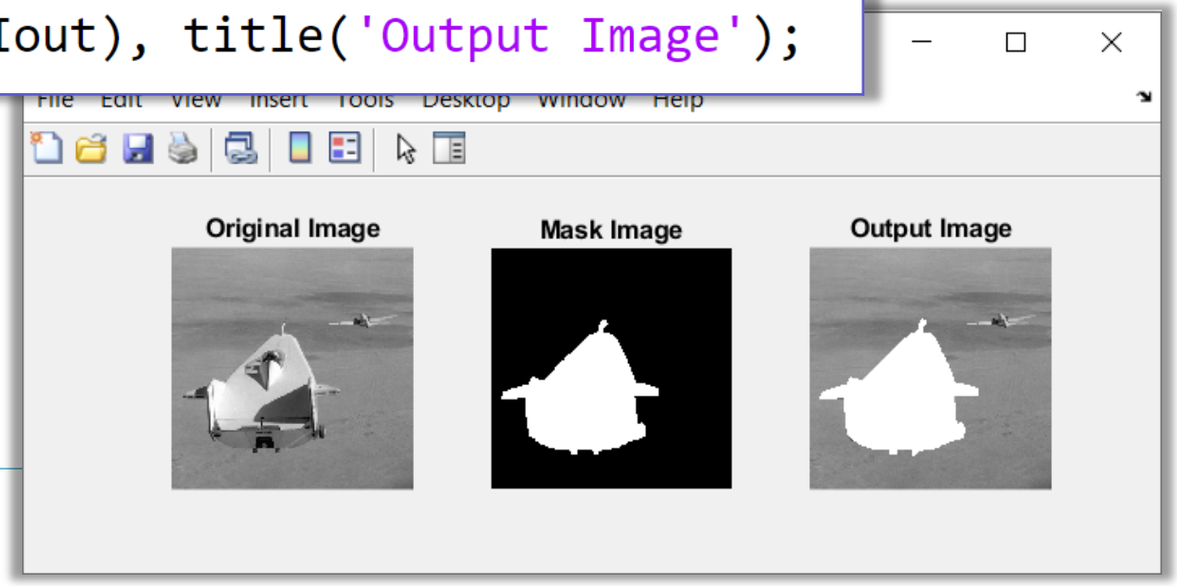


Logical Operations on Images – MATLAB

- The **MATLAB**'s IPT includes function called **bitor** that can be used to compute the logic OR between two images.

- **Example:**

```
Img = imread('liftingbody.png');
Mask = roipoly(Img);
Iout = bitor(Img, im2uint8(Mask));
figure
subplot(1,3,1), imshow(Img), title('Original Image');
subplot(1,3,2), imshow(Mask), title('Mask Image');
subplot(1,3,3), imshow(Iout), title('Output Image');
```

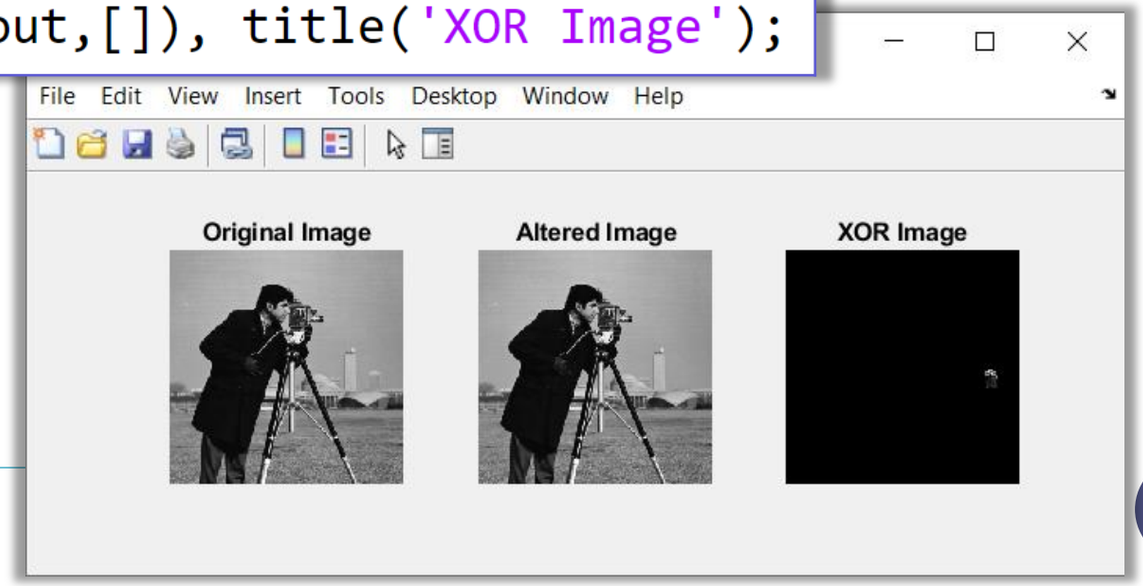


Logical Operations on Images – MATLAB

- The **MATLAB**'s IPT includes function called **bitxor** that can be used to find the differences between two images.

- **Example:**

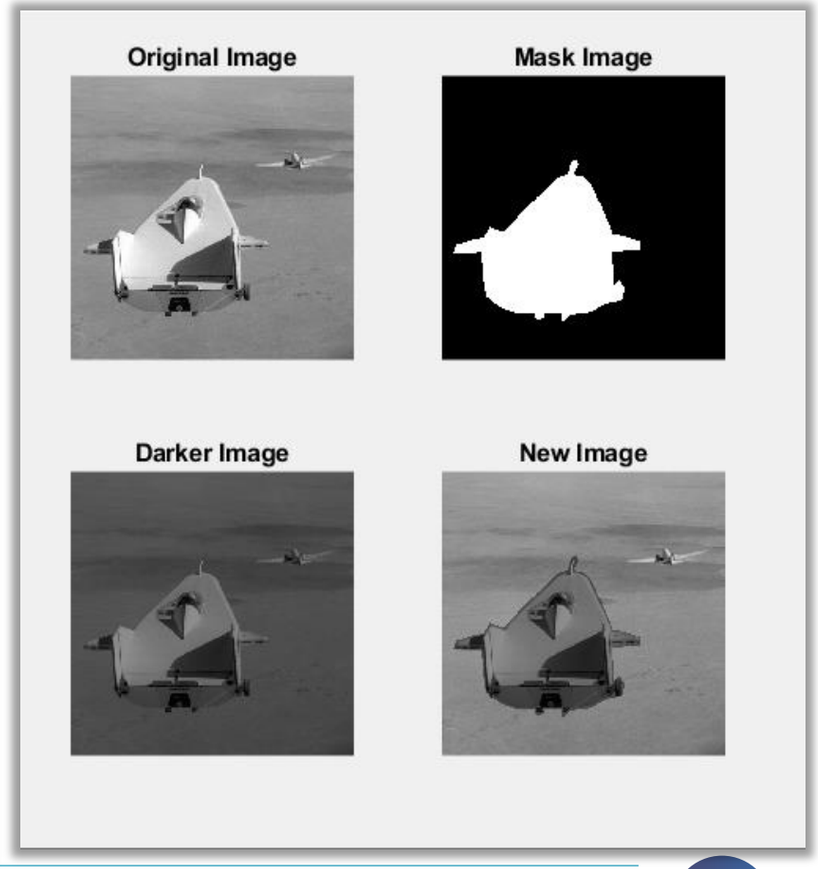
```
IA = imread('cameraman.tif');
IB = imread('cameraman2.tif');
Iout = bitxor(IA, IB);
figure
subplot(1,3,1), imshow(IA), title('Original Image');
subplot(1,3,2), imshow(IB), title('Altered Image');
subplot(1,3,3), imshow(Iout,[]), title('XOR Image');
```



Logical Operations on Images – MATLAB

- **Example:** Logic operators are often combined to achieve a particular task. Use logic operators to darken an image only within a region of interest, while displaying the original image elsewhere.

```
IA = imread('liftingbody.png');
Mask = im2uint8(roipoly(IA));
IB = imdivide(IA,1.8);
Msk_Cmp = imcomplement(Mask);
OB = bitand(IB, Mask);
BG = bitand(IA, Msk_Cmp);
New_img = bitor(OB, BG);
figure
subplot(1,3,1), imshow(IA), title('Original Image');
subplot(1,3,2), imshow(IB), title('Darker Image');
subplot(1,3,3), imshow(New_img), title('New Image');
```



Thank You